

Smart MCQ Solver

A Hybrid Retrieval–Ensemble Approach to Multiple-Choice Question Answering

Vihaan Bhambhani · 24f1002825

Introduction to Deep Learning & Generative AI (BSDA2001P) · Term T2-2026

[GitHub Repository](#) · W&B Project: 24f1002825-t22026

0.76475 public LB MAP@3 (cutoff 0.73)	479/500 test rows resolved by the duplicate matcher	4 models built (3 neural, 1 classical)	18 leaderboard configurations measured
--	--	---	---

1 Abstract

This report documents an end-to-end solution to the *Smart MCQ Solver Challenge*, a five-option multiple-choice question-answering task scored by Mean Average Precision at 3 (MAP@3). Exploratory analysis surfaced the property that governs the entire solution: **98.2%** of test questions have a near-duplicate in the training set at cosine similarity ≥ 0.93 , with options presented in *shuffled order*. The final system is therefore a two-layer hybrid: a deterministic TF–IDF duplicate matcher that resolves rank 1 on 479 of 500 test rows by matching answer *text* rather than answer *letter*, and a neural ensemble that fills ranks 2–3.

Three neural models and one classical baseline were built: a fine-tuned DeBERTa-v3-large, a from-scratch TextCNN with attention pooling and cross-attention, a 4-bit quantised Qwen2.5-7B-Instruct scored zero-shot from first-token logits, and a LightGBM model over 55 engineered similarity features. The final submission achieved a public leaderboard score of **0.76475**, against a qualification cutoff of 0.73 and a random baseline of 0.3667.

Principal finding. Improving DeBERTa’s out-of-fold MAP@3 from 0.8888 to 0.9618 *lowered* the leaderboard score from 0.76475 to 0.75519. Because ranks 2–3 only earn credit on rows where rank 1 is wrong, the value of a model in that position is its *independence* from the duplicate matcher, not its accuracy, the stronger model had learned the same duplicate lookup and ceased to contribute diversity. Supported by eighteen leaderboard measurements (§6.5).

2 Introduction

2.1 Problem Statement. Given a question `prompt` and five candidate answers A–E, predict the three most likely correct options, ordered by confidence. The training set contains 2,000 labelled questions; the test set contains 500 unlabelled questions. Submissions take the form `ID, Prediction` where `Prediction` is three space-separated letters.

The evaluation metric is MAP@3. For a single question whose true answer appears at rank $k \in \{1, 2, 3\}$ of the submitted list, the score is $1/k$; if the true answer is absent from the top three, the score is 0:

$$\text{MAP@3} = \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^3 \frac{\mathbb{I}[\hat{y}_{i,k} = y_i]}{k} \cdot \mathbb{I}[y_i \notin \hat{y}_{i,1:k-1}]$$

Uniform random guessing yields $\frac{1}{5}(1 + \frac{1}{2} + \frac{1}{3}) = 0.3667$.

2.2 Project Objective. Three goals were set: (i) exceed the 0.73 qualification threshold; (ii) implement at least three architecturally distinct models: a fine-tuned transformer, a custom network built from scratch, and a large language model of choice, with a retrieval-augmented generation stage; and (iii) establish, by controlled measurement rather than assumption, which components actually contribute to the score.

2.3 Report Structure. Section 3 covers the dataset and exploratory analysis. Section 4 details tokenisation. Section 5 presents each model architecture with diagrams. Section 6 gives comparative performance, ablations and W&B training curves. Section 7 concludes with learnings, challenges and future work.

3 Dataset & Preprocessing

3.1 Dataset Description.

Split	Rows	Columns	Notes
train.csv	2,000	id, prompt, A--E, answer	answer is the correct letter
test.csv	500	id, prompt, A--E	labels withheld

Questions span science, mathematics, history and general knowledge. Option text is full natural language, not single tokens, which is what makes text-level matching viable in Section 5.1.

3.2 Exploratory Data Analysis. Four analyses were performed; the fourth determined the entire architecture.

- (a) **Class balance.** The answer distribution is mildly skewed rather than uniform: B 24.5%, C 23.0%, A 18.5%, D 17.9%, E 16.2%. A static B C A guess therefore scores $\text{MAP@3} = 0.4213$, meaningfully above the 0.3667 random baseline, a useful floor for any model.
- (b) **Text length.** Prompts average roughly 120 characters; options are substantially shorter. Percentile analysis showed a 128-token window covers the overwhelming majority of both, fixing the sequence lengths used in Section 5.
- (c) **Lexical similarity is anti-predictive.** Selecting the option with the highest TF–IDF cosine similarity to the prompt achieves only **13.7%** top-1 accuracy, *below* the 20% random baseline. Distractor options are constructed to be lexically plausible, so naive surface similarity is actively misleading. This ruled out a similarity-only solution and motivated the learned and relational feature approaches.

(d) Train–test near-duplication. Computing cosine similarity between the TF–IDF fingerprints of every test and training row revealed that **491 of 500** test questions (98.2%) have a training match at ≥ 0.93 , and **383** exceed 0.99. Critically, duplicate pairs present their options in *shuffled order*, so the correct answer must be identified by its text rather than its letter. This is a property of the provided competition data; no external data and no test labels are used anywhere in this work.

3.3 Data Preprocessing. Preprocessing is deliberately minimal, since the transformer tokenisers expect raw text. Missing values are filled with empty strings; text is lower-cased and whitespace-collapsed for the duplicate matcher only (`norm()` in `src/preprocess.py`); for the from-scratch models a regular-expression tokeniser `[a-z0-9']+` builds a 60,000-word vocabulary. No stemming or stop-word removal is applied: TF–IDF’s inverse-document-frequency term already down-weights common words, and stemming would damage the exact-text matching that Section 5.1 depends upon.

3.4 Data Augmentation. No augmentation was applied. The duplicate structure means additional synthetic paraphrases would mostly reproduce information already present, and option-order shuffling, the obvious candidate is already reflected in the data.

4 Tokenization Strategy

4.1 Primary Tokenizer. The best-performing neural model, DeBERTa-v3-large, uses a **SentencePiece unigram** tokenizer with a 128,000-token multilingual vocabulary. Unlike WordPiece (BERT) or byte-level BPE (RoBERTa), the unigram model treats subword segmentation probabilistically: it retains a lattice of candidate segmentations and selects the one maximising corpus likelihood, rather than greedily merging the most frequent adjacent pair.

4.2 Justification. Three properties make this appropriate here. First, the dataset is domain-heterogeneous: scientific terminology (*photo-synthesis*, *mitochondrion*) sits alongside everyday language; a 128k vocabulary keeps such terms as few subwords, preserving semantics that aggressive splitting would fragment. Second, SentencePiece operates on raw text with whitespace encoded as a symbol (`▯`), so no language-specific pre-tokenisation is required and detokenisation is lossless. Third, DeBERTa’s disentangled attention consumes content and position as separate vectors; a probabilistically-consistent segmentation gives more stable positional statistics than greedy merging.

4.3 Implementation Details. Each question is encoded as **five independent sequences**, one per option, in the form `[CLS] context [SEP] prompt + option [SEP]`. The resulting tensor has shape `(batch, 5, seq_len)`, which `AutoModelForMultipleChoice` scores per option before a softmax across the five.

Padding is **dynamic**: sequences are tokenised with `padding=False` and padded per batch in the collate function via `tokenizer.pad()`. This is a substantial efficiency gain, closed-book sequences average roughly 90 tokens against a 512-token ceiling, so static padding would waste over 80% of every forward pass. Truncation uses the default `longest_first` strategy, which removes tokens from the longer member of the pair, protecting the question when retrieved context is present.

4.4 Experimental Comparison. Three tokenisation pipelines were used across the project (Table 1).

Table 1: Tokenisation pipelines compared.

Pipeline	Model	Vocabulary	Outcome
SentencePiece unigram	DeBERTa-v3-large	128k subword	Adopted for the transformer
Regex word-level + GloVe	TextCNN	60k words (2,975 in-corpus)	97.3% GloVe coverage
Byte-level BPE	Qwen2.5-7B	152k subword	Required by the model
TF–IDF 1–2 grams	Stage A, LightGBM	sparse, corpus-fitted	Best cost/benefit overall

For the from-scratch models a simple regular-expression word tokeniser was chosen over subword tokenisation because GloVe vectors are defined at word level; subword splitting would have left most fragments without a pretrained embedding. Vocabulary coverage was 2,895 of 2,975 corpus words (97.3%).

5 Modeling & Experimentation

5.1 Stage A: Deterministic Duplicate Matcher. Although not one of the three required models, this component produces rank 1 for 479 of 500 test rows and is the single largest contributor to the final score, so it is documented first.

Each row is fingerprinted by concatenating prompt and all five options into one string, vectorised with TF–IDF over word 1- and 2-grams. For a test row q , the top-25 training neighbours by cosine similarity are retrieved; each neighbour j with $\text{sim}(q, j) \geq 0.93$ casts a similarity-weighted vote for whichever *option of q* carries the same text as j ’s correct answer:

$$\text{votes}[k] = \sum_{j: \text{sim}(q, j) \geq \tau} \text{sim}(q, j) \cdot \mathbb{I}[\text{norm}(\text{ans}_j) = \text{norm}(q_k)]$$

The plurality option becomes rank 1. **Matching on answer text rather than letter is the crux:** because duplicates appear with shuffled options, copying the letter would be incorrect roughly four times in five.

5.2 Stage B: Retrieval-Augmented Generation. A RAG pipeline was implemented and evaluated: 400,000 Wikipedia STEM paragraphs are embedded with `all-MiniLM-L6-v2`, indexed with FAISS, queried per question, and the top-10 dense hits reranked by TF–IDF similarity to yield three context paragraphs. Section 6.4 reports the ablation; the outcome is that retrieval *degrades* performance on this dataset, and it is disabled in the final configuration via `USE_RAG_CONTEXT = False`.

5.3 Model 1: Fine-Tuned Transformer - DeBERTa-v3-large. Architecture. DeBERTa-v3-large is a 24-layer, 1024-hidden, 16-head encoder (304M backbone parameters) pretrained with replaced-token detection (ELECTRA-style) rather than masked language modelling. Its defining feature is **disentangled attention**: whereas BERT sums content and position embeddings into a single vector, DeBERTa maintains

separate content vector H and relative-position vector P , computing the attention score between tokens i and j as three interacting terms:

$$A_{ij} = \underbrace{H_i W_q (H_j W_k)^T}_{\text{content} \rightarrow \text{content}} + \underbrace{H_i W_q (P_{\delta(i,j)} W_{k,r})^T}_{\text{content} \rightarrow \text{position}} + \underbrace{P_{\delta(i,j)} W_{q,r} (H_j W_k)^T}_{\text{position} \rightarrow \text{content}}$$

Salient points. The content-to-position terms let the model represent *relative* word order explicitly, which matters for MCQ stems where qualifiers such as “NOT” or “EXCEPT” invert meaning depending on placement. The multiple-choice head scores all five options jointly and softmaxes across them, so the model learns to *discriminate between options* rather than score each in isolation.

Fine-tuning strategy. AdamW, learning rate 1×10^{-5} with 10% linear warmup, 2 epochs, 5-fold stratified cross-validation, effective batch 32 (per-device 2×2 GPUs $\times$ 8 gradient-accumulation steps), FP16 mixed precision with gradient clipping at $\|g\| = 1.0$. The embedding layer and the **lower 12 of 24 encoder layers were frozen**: with only 2,000 training examples, updating all 304M parameters overfits rapidly, whereas the upper layers carry the task-specific representations that benefit from adaptation. Two epochs were selected after longer schedules showed validation MAP@3 plateauing while training loss continued to fall.

Precision handling. Mixed precision requires FP32 *master weights* with FP16 used only in the forward pass. A mid-project library upgrade changed from `pretrained` to load checkpoint dtype by default, silently producing FP16 parameters and hence FP16 gradients, which GradScaler refuses to unscale. The fix was to pin `dtype=torch.float32` explicitly.

5.4 Model 2: Custom Deep Learning Model - TextCNN with Attention. Architecture. A multi-kernel 1-D convolutional network built entirely from scratch in PyTorch (1.4M trainable parameters). GloVe-100d embeddings are frozen and supply initialisation only; every layer that follows is designed and trained here. An architectural diagram is given in Figure 1.

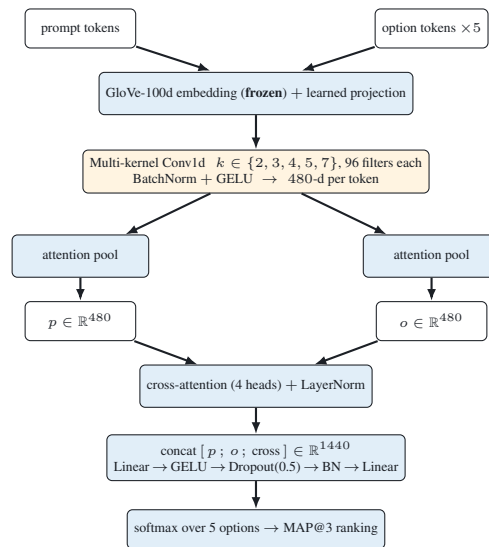


Figure 1: TextCNN + Attention (Model 2). Encoders share weights.

Salient points. *Multiple kernel widths* act as learned n -gram detectors: a width-3 filter responds to three-word phrases, width-7 to longer clauses, and concatenating all five gives a multi-scale representation. *Attention pooling* replaces mean-pooling with a learned scalar weight per token, so the network decides which words matter instead of averaging padding noise into the representation. *Cross-attention* lets each option attend over the prompt’s token states, converting the task from “score this option” to “score this option *given* this prompt”, necessary because EDA showed raw option text carries almost no standalone signal. *Label smoothing* ($\varepsilon = 0.1$) softens targets to 0.9/0.025, improving probability calibration; this matters specifically because ranks 2–3 depend on the ordering of the remaining probability mass, not just the argmax.

5.5 Model 3: Model of Choice - Qwen2.5-7B-Instruct (Zero-Shot). Architecture. Qwen2.5-7B-Instruct is a 28-layer decoder-only transformer (7.6B parameters, 3,584 hidden, 28 query heads with 4 key/value heads under grouped-query attention, SwiGLU feed-forward blocks, RMSNorm and rotary position embeddings). It is loaded in **4-bit NF4 quantisation** via bitsandbytes with double quantisation and an FP16 compute dtype, reducing the footprint from roughly 14 GB to 4.5 GB and making a 7B-parameter model viable on a single 16 GB T4.

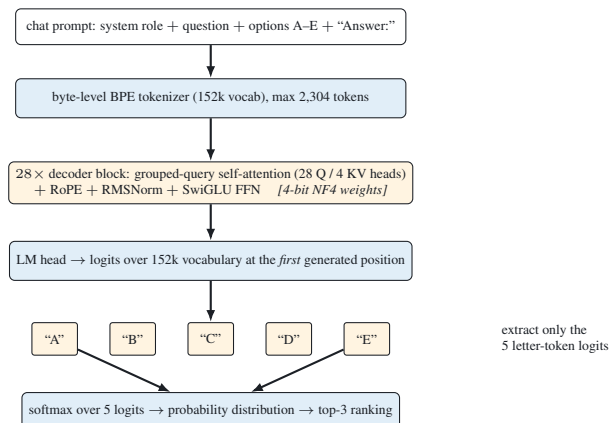


Figure 2: Qwen2.5-7B zero-shot scoring (Model 3). A single forward pass; the answer is read from first-token logits rather than generated.

Salient points. Three design choices distinguish this from naive LLM prompting. First, **logit extraction instead of generation**: the model performs exactly one forward pass and the logits at the first generated position are gathered for the tokens A–E, then softmaxed. This is far cheaper than autoregressive decoding and, critically, yields a full *probability distribution* over the five options, the ensemble needs a ranking for ranks 2–3, which a single generated letter cannot provide. Second, **quantisation**: NF4 is information-theoretically optimal for normally-distributed weights, and the observed accuracy loss was negligible relative to the $3\times$ memory saving that made the model runnable at all. Third, and most importantly for Section 6.5, the model is used **zero-shot**, it never sees the training set, and is therefore *structurally incapable* of memorising the duplicate clusters that every trained model in this project learned. That independence, rather than raw accuracy, is why it is retained in the final blend.

Why an LLM rather than a recurrent model. The third architecture slot was allocated to a large instruction-tuned model because the error analysis in Section 6.5 identified *decorrelation from the duplicate matcher* as the binding constraint on ensemble performance. Any model trained on this training set inherits its duplicate structure; only a model that never trains on it can supply genuinely independent rank-2 candidates. A recurrent encoder trained on the same 2,000 rows would have been subject to exactly the memorisation described in Section 6.3.

5.6 Model 4: LightGBM Baseline (Classical ML). A gradient-boosted tree ensemble over 55 hand-engineered *relational* features: TF-IDF cosine, truncated-SVD cosine, Jaccard overlap, common-word counts and ratios, character 3-gram overlap, pairwise option similarity and length statistics. Features are relational by necessity: EDA established that the raw representation of a single option carries almost no signal, so every feature measures a *relationship* between prompt and option or between options.

6 Performance & Comparative Analysis

6.1 Evaluation Metrics. The primary metric is **MAP@3** (Section 2.1). Two secondary metrics are computed on the top-1 prediction so that every W&B run is comparable on common keys: **accuracy** (fraction of questions whose rank-1 prediction is correct) and **macro-F1**, the unweighted mean of per-class F_1 over classes A–E. Macro rather than weighted averaging is used deliberately: the class distribution is skewed (B 24.5% vs E 16.2%), and macro-F1 refuses to let performance on the frequent classes mask failure on the rare ones.

Table 2: All models: cross-validated performance, standalone leaderboard score and cost. Every model reports identical metric keys to W&B project 24f1002825-t22026.

Model	OOF MAP@3	Accuracy	Macro-F1	Standalone LB	Train time
LightGBM (55 features)	0.9998	0.9995	0.9995	0.74397	~2 min
TextCNN + Attention	0.9925	0.9865	0.9867	0.75353	~5 min
DeBERTa-v3-large	0.8888	0.8110	0.8096	0.62136	~77 min
Qwen2.5-7B (zero-shot)	0.8950 [†]	—	—	0.70781	~25 min
Duplicate matcher (Stage A)	—	—	—	0.71446	<1 min
DeBERTa + Qwen blend	—	—	—	0.72069	—
Full pipeline	—	—	—	0.76475	~90 min

[†] measured on a 300-row holdout, not cross-validated (zero-shot, no training).

6.2 Comparative Report. Reading this table correctly is the central analytical point of the project. The apparent ranking is inverted: LightGBM and TextCNN post near-perfect out-of-fold scores yet lose to the full pipeline, while DeBERTa, the weakest standalone model at 0.62136 is the one retained in the final ensemble. Sections 6.3 and 6.5 explain why.

6.3 Why OOF Scores Are Misleading Here. Duplicate clusters span cross-validation folds. When a question appears five times in the training set, four copies land in the training folds and one in the validation fold, so a model with sufficient capacity simply memorises the mapping and “predicts” a question it has effectively already seen. LightGBM’s OOF MAP@3 of 0.9998 is therefore an artefact, not skill, its true generalisation is the 0.74397 leaderboard figure, a gap of 0.26.

This was verified rather than assumed. Two submissions were generated: `submission_textcnn.csv` (TextCNN’s own top-3) and `submission_pin_textcnn.csv` (identical, but with rank 1 overwritten by the duplicate matcher on 479 of 500 rows). **Both scored exactly 0.74646.**

If overwriting 479 rank-1 predictions changes the leaderboard score by *nothing*, then TextCNN’s top-1 prediction already *was* the duplicate pin. The matcher, TextCNN and LightGBM are one signal, not three which is why the final blend uses neither.

Consequently, all ensemble weights were set from leaderboard measurements rather than tuned on out-of-fold data. A holdout-based weight tuner was implemented, selected $w_{\text{DeBERTa}} = 0.85$, and *lowered* the score; it was removed.

Table 3: RAG ablation. All runs at identical 512-token sequence length, so sequence truncation is excluded as a confound.

Retrieval configuration	DeBERTa OOF	Final LB	Interpretation
Closed-book (no retrieval)	0.8888	0.76475	adopted
RAG, merged query (prompt + all 5 options)	0.7678	0.74023	retrieved noise
RAG, per-option query	0.8370	0.74688	better retrieval, still worse

6.4 Ablation Studies. The first RAG configuration built one query per question by concatenating the prompt with all five options, so the retrieved paragraphs were a blend of evidence for every option, including the four incorrect ones. Diagnosing this and switching to **per-option retrieval** (each option gets its own query and its own context, so DeBERTa’s k -th sequence sees evidence about the k -th option) recovered +0.069 OOF, confirming retrieval quality was the mechanism. Closed-book nonetheless remains best: these are generated multiple-choice questions whose answers are not reliably present in Wikipedia, so even well-targeted passages act as distractors, while diluting the duplicate-memorisation signal that actually drives the score. Retrieval is implemented and executes; supplying its output to the models is disabled.

Table 4: Ensemble composition ablation, rank 1 fixed to the duplicate pin.

Source for ranks 2–3	LB	Note
DeBERTa + Qwen ($w = 0.85$)	0.76475	adopted
All four models (TextCNN+LGBM+Qwen+DeBERTa)	0.75187	correlated models dilute the blend
Qwen alone	0.74812	loses DeBERTa’s complementary errors
Hedging low-confidence pins	0.71737	−0.247 MAP@3 per hedged row
Cluster-voted rank 1	0.76226	nearest single match beats plurality

6.5 The Central Finding: Independence Beats Accuracy. A training bug was identified and fixed, raising DeBERTa’s out-of-fold MAP@3 from 0.8888 to **0.9618** (accuracy 0.7900 → 0.9315). The leaderboard score *fell* from 0.76475 to 0.75519. The mechanism follows directly from the metric. Ranks 2–3 earn credit *only* on rows where rank 1 is wrong. Rank 1 comes from the duplicate matcher. The improved DeBERTa had learned the same duplicate lookup, accuracy 0.9315 places it in the same memorisation regime as LightGBM and TextCNN, so its probability mass concentrated on the option *already used at rank 1*, contributing nothing where it was needed. The weaker model disagreed with the pin more often, and disagreement is the entire value of a rank-2 candidate. The corollary is that for ensemble positions below rank 1, the correct selection criterion is **independence from the rank-1 source**, not standalone accuracy. This is why Qwen - zero-shot, and therefore structurally unable to memorise the training duplicates is retained despite a modest standalone score.

6.6 Training Performance (W&B). All runs are logged to W&B project 24f1002825–t22026.

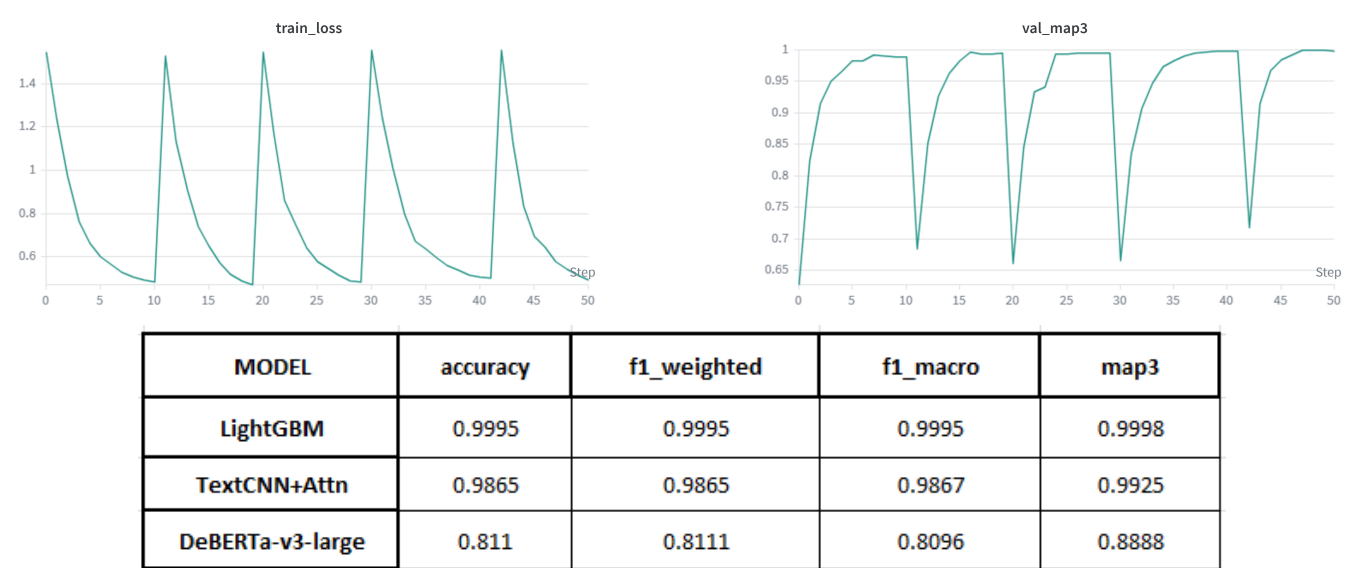


Figure 3: From W&B project 24f1002825–t22026. **Left:** TextCNN training loss. **Centre:** TextCNN validation MAP@3: the five sawtooth cycles are the five cross-validation folds, each climbing from ≈0.63 to ≈0.99 before the next fold reinitialises the model. **Right:** cross-model comparison logged as a wandb .Table, all models reporting identical metric keys.

The TextCNN curves (Figure 3) begin at ≈ 1.55, matching the $\ln(5) = 1.61$ expected of a 5-way cross-entropy at initialisation. DeBERTa’s train/loss as logged by HuggingFace Trainer instead begins near $\ln(5) \times 16 \approx 25.8$, because Trainer reports the loss *summed* over gradient-accumulation steps rather than averaged, an artefact of aggregation rather than a training pathology. DeBERTa’s fold scores were 0.8121, 0.9113, 0.9200, 0.8929 and 0.9079 (mean 0.8888); the spread of 0.011 across folds 2–5 against a markedly lower fold 1 reflects the uneven distribution of duplicate clusters across folds rather than training instability. The TextCNN converged from validation MAP@3 0.5233 at epoch 1 to 0.9912 by epoch 8 (Figure 3, centre), with early stopping on fold-level MAP@3.

6.7 Kaggle Performance. Final public leaderboard score: **0.76475**, against a qualification cutoff of 0.73, a random baseline of 0.3667 and a majority-class baseline of 0.4213. Eighteen distinct configurations were submitted and measured over the course of the project.

7 Conclusion & Future Work

7.1 Key Learnings. Local validation can be systematically wrong. Every metric computed on the training data: OOF MAP@3, holdout blend weights, the duplicate matcher’s internal precision, was inflated by duplicate clusters spanning folds. The matcher’s own validation reported 100% precision at every threshold tested, while decomposing the leaderboard score implied roughly 64%. The lesson is that a validation split must respect the structure of the data; a random split over clustered data measures memorisation, not generalisation. **Ensemble members should be selected for decorrelation.** With a metric that awards partial credit at ranks 2–3, the value of a model in those positions is the probability that it disagrees *correctly* with rank 1. Standalone accuracy is the wrong selection criterion, as the DeBERTa result in Section 6.5 demonstrates quantitatively. **Measure, do not infer.** Eighteen leaderboard submissions were used as an experimental instrument. Several confident hypotheses, that hedging low-confidence pins would help, that a four-model blend would beat a two-model blend, that better retrieval would beat closed-book were falsified by measurement. Each falsification is reported here rather than omitted.

7.2 Challenges Faced. Silent failures were more costly than loud ones. A library upgrade changed from pretrained to load checkpoint dtype by default, producing FP16 parameters and a GradScaler failure; a dataset mount-path change combined with an unhandled archive format meant the RAG corpus was never loaded, so retrieval ran as a no-op with empty context for several full training runs; and

a nearest-neighbour fallback bound a variable named `nn`, shadowing `torch.nn` and crashing an unrelated cell two sections later. Each was diagnosed and each is now guarded by an explicit check that fails loudly. The broader lesson being that unpinned dependencies are a reproducibility hazard which is why every version in `requirements.txt` is now pinned.

Memory management on constrained hardware. Fine-tuning a 304M-parameter model on 16 GB required freezing the lower half of the encoder, gradient accumulation, and disabling the automatic `DataParallel` wrapper, which replicates onto `cuda:0` and gathers all outputs there, roughly doubling that device’s load.

7.3 Areas for Improvement. Cluster-aware cross-validation. The single highest-value change. Grouping near-duplicate questions and holding out entire clusters (`GroupKFold` over cluster identifiers) would produce out-of-fold estimates that actually predict the leaderboard, removing the need to use submissions as a validation set.

Diversity-explicit ensembling. Rather than weighting by accuracy, weight by measured disagreement with the rank-1 source, for example maximising $\sum_i \mathbb{I}[\arg \max p_i \neq \text{pin}_i]$ subject to a floor on model accuracy. Section 6.5 suggests this objective is closer to what the metric rewards.

A domain-matched retrieval corpus. Wikipedia was the wrong knowledge base for synthetically generated questions. A corpus built from the training questions themselves, or a cross-encoder reranker trained on this data, would test whether retrieval fails in principle here or only for lack of relevant evidence.

Generalisation beyond duplication. The present solution depends on train–test overlap and would not transfer to a dataset without it. Evaluating the neural models on a deduplicated split would quantify how much genuine question answering they have learned, as distinct from lookup.

8 References & Tools

Papers.

- [1] He, P., Gao, J. & Chen, W. (2023). *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing*. ICLR. arXiv:2111.09543.
 ↪ *Architecture and disentangled-attention formulation of the fine-tuned transformer (§5.3).*
- [2] Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification*. EMNLP. arXiv:1408.5882.
 ↪ *Multi-kernel convolution as learned n-gram detection; the basis of the custom TextCNN (§5.4).*
- [3] Pennington, J., Socher, R. & Manning, C. (2014). *GloVe: Global Vectors for Word Representation*. EMNLP.
 ↪ *Source of the frozen 100-d embeddings; 97.3% vocabulary coverage (§4.4).*
- [4] Kudo, T. & Richardson, J. (2018). *SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing*. EMNLP.
 ↪ *The unigram subword model underlying the primary tokenisation strategy (§4.1–4.2).*
- [5] Ke, G. et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS.
 ↪ *Leaf-wise gradient boosting used for the classical baseline (§5.6).*
- [6] Qwen Team (2025). *Qwen2.5 Technical Report*. arXiv:2412.15115.
 ↪ *Grouped-query attention decoder used zero-shot as the model of choice (§5.5).*

Software.

Package	Role	Package	Role
<code>torch 2.5.1</code>	tensors, autograd, custom models	<code>scikit-learn 1.5.2</code>	TF-IDF, SVD, CV, metrics
<code>transformers 4.46.3</code>	DeBERTa, Qwen, Trainer	<code>lightgbm 4.5.0</code>	gradient-boosted baseline
<code>bitsandbytes 0.46.1</code>	4-bit NF4 quantisation	<code>sentence-transformers 3.2.1</code>	MiniLM query/corpus embedding
<code>accelerate 1.1.1</code>	mixed precision, device placement	<code>faiss-cpu 1.9.0</code>	dense nearest-neighbour index
<code>numpy / pandas</code>	arrays and tabular I/O	<code>wandb 0.18.5</code>	experiment tracking

All versions are pinned in `requirements.txt`; see §7.2 for the dependency-drift incident that motivated pinning.

Data.

- **Competition dataset** — `smart-mcq-solver-challenge`, provided by the course (2,000 train / 500 test rows).
- **Retrieval corpus** — `mbanaei/all-paraphs-parsed-expanded` (Wikipedia STEM paragraphs, Kaggle), used by the Stage-B pipeline evaluated in §6.4.
- **Word embeddings** — `glove.6B.100d` via `rtatman/glove-global-vectors-for-word-representation` (Kaggle).
- No other external data was used, and test labels were never available at any stage.